## 1. Importing Libraries

```
1 import pandas as pd
2 import numpy as np
3 import sklearn
4 from sklearn.model_selection import train_test_split
5 import sklearn.preprocessing
6 from sklearn.preprocessing import StandardScaler
7 import matplotlib.pyplot as plt
```

## 2. Importing dataset

```
1 df = pd.read_csv("/content/buddymove_holidayiq.csv")
2 df
```

| | User Id | Sports | Religious | Nature | Theatre | Shopping | Picnic |
|---|---|---|---|---|---|---|---|
| **0** | User 1 | 2 | 77 | 79 | 69 | 68 | 95 |
| **1** | User 2 | 2 | 62 | 76 | 76 | 69 | 68 |
| **2** | User 3 | 2 | 50 | 97 | 87 | 50 | 75 |
| **3** | User 4 | 2 | 68 | 77 | 95 | 76 | 61 |
| **4** | User 5 | 2 | 98 | 54 | 59 | 95 | 86 |
| **...** | ... | ... | ... | ... | ... | ... | ... |
| **244** | User 245 | 18 | 139 | 148 | 129 | 129 | 168 |
| **245** | User 246 | 22 | 114 | 228 | 104 | 84 | 168 |
| **246** | User 247 | 20 | 124 | 178 | 104 | 158 | 174 |
| **247** | User 248 | 20 | 133 | 149 | 139 | 144 | 213 |
| **248** | User 249 | 20 | 143 | 149 | 139 | 159 | 143 |

249 rows × 7 columns

Saved successfully!

## 3. Taking care of missing data

*a) isnull()/ notnull()- Both function help in checking whether a value is NaN or not.*

```
1 df.isnull()
```

| | User Id | Sports | Religious | Nature | Theatre | Shopping | Picnic |
|---|---|---|---|---|---|---|---|
| **0** | False | False | False | False | False | False | False |
| **1** | False | False | False | False | False | False | False |
| **2** | False | False | False | False | False | False | False |
| **3** | False | False | False | False | False | False | False |
| **4** | False | False | False | False | False | False | False |
| **...** | ... | ... | ... | ... | ... | ... | ... |
| **244** | False | False | False | False | False | False | False |
| **245** | False | False | False | False | False | False | False |
| **246** | False | False | False | False | False | False | False |
| **247** | False | False | False | False | False | False | False |

```
1 print(df.isnull().sum())
```

```
User Id      0
Sports       0
Theatre      0
Shopping     0
Picnic       0
dtype: int64
```

Saved successfully!

*As there are no missing values we will change some values to NAN for the purpose of demonstration*

```
1 df=df.replace(59,np.NaN)
2 df=df.replace(76,np.NaN)
3 df
4 print(df.isnull().sum())
```

```
User Id      0
Sports       0
Religious    4
Nature       6
Theatre      4
Shopping     5
Picnic       2
dtype: int64
```

```
1 df.isnull()
```

| | User Id | Sports | Religious | Nature | Theatre | Shopping | Picnic |
|---|---|---|---|---|---|---|---|
| **0** | False | False | False | False | False | False | False |
| **1** | False | False | False | True | True | False | False |
| **2** | False | False | False | False | False | False | False |
| **3** | False | False | False | False | False | True | False |
| **4** | False | False | False | False | True | False | False |
| **...** | ... | ... | ... | ... | ... | ... | ... |
| **244** | False | False | False | False | False | False | False |
| **245** | False | False | False | False | False | False | False |
| **246** | False | False | False | False | False | False | False |
| **247** | False | False | False | False | False | False | False |

```
1 bool_series = pd.notnull(df)
2 bool_series
```

| | User Id | Sports | Religious | Nature | Theatre | Shopping | Picnic |
|---|---|---|---|---|---|---|---|
| | | | | e | True | True | True | True |
| | | | | | False | False | True | True |
| **2** | True | True | True | True | True | True | True |
| **3** | True | True | True | True | True | False | True |
| **4** | True | True | True | True | False | True | True |
| **...** | ... | ... | ... | ... | ... | ... | ... |
| **244** | True | True | True | True | True | True | True |
| **245** | True | True | True | True | True | True | True |
| **246** | True | True | True | True | True | True | True |
| **247** | True | True | True | True | True | True | True |
| **248** | True | True | True | True | True | True | True |

249 rows × 7 columns

Saved successfully!

```
Filling missing values using fillna(), replace() and interpolate()
In order to fill null values in a datasets, we use fillna(), replace() and
interpolate() function these function replace NaN values with some value of
their own.
```

Filling missing values using fillna(), replace() and interpolate() In order to fill null values in a datasets, we use fillna(), replace() and interpolate() function these function replace NaN values

with some value of their own.

Filling null values with a single value

```
1 df.fillna(0)
```

| | User Id | Sports | Religious | Nature | Theatre | Shopping | Picnic |
|---|---|---|---|---|---|---|---|
| **0** | User 1 | 2 | 77.0 | 79.0 | 69.0 | 68.0 | 95.0 |
| **1** | User 2 | 2 | 62.0 | 0.0 | 0.0 | 69.0 | 68.0 |
| **2** | User 3 | 2 | 50.0 | 97.0 | 87.0 | 50.0 | 75.0 |
| **3** | User 4 | 2 | 68.0 | 77.0 | 95.0 | 0.0 | 61.0 |
| **4** | User 5 | 2 | 98.0 | 54.0 | 0.0 | 95.0 | 86.0 |
| **...** | ... | ... | ... | ... | ... | ... | ... |
| **244** | User 245 | 18 | 139.0 | 148.0 | 129.0 | 129.0 | 168.0 |
| **245** | User 246 | 22 | 114.0 | 228.0 | 104.0 | 84.0 | 168.0 |
| **246** | User 247 | 20 | 124.0 | 178.0 | 104.0 | 158.0 | 174.0 |
| | | | .0 | 149.0 | 139.0 | 144.0 | 213.0 |
| **248** | User 249 | 20 | 143.0 | 149.0 | 139.0 | 159.0 | 143.0 |

249 rows × 7 columns

Saved successfully!

Filling null values with the previous ones

```
1   df.fillna(method ='pad')
```

| | User Id | Sports | Religious | Nature | Theatre | Shopping | Picnic |
|---|---|---|---|---|---|---|---|

Filling null value with the next ones

```
1 df.fillna(method ='bfill')
```

| | User Id | Sports | Religious | Nature | Theatre | Shopping | Picnic |
|---|---|---|---|---|---|---|---|
| **0** | User 1 | 2 | 77.0 | 79.0 | 69.0 | 68.0 | 95.0 |
| **1** | User 2 | 2 | 62.0 | 97.0 | 87.0 | 69.0 | 68.0 |
| **2** | User 3 | 2 | 50.0 | 97.0 | 87.0 | 50.0 | 75.0 |
| **3** | User 4 | 2 | 68.0 | 77.0 | 95.0 | 95.0 | 61.0 |
| **4** | User 5 | 2 | 98.0 | 54.0 | 93.0 | 95.0 | 86.0 |
| **...** | ... | ... | ... | ... | ... | ... | ... |
| **244** | User 245 | 18 | 139.0 | 148.0 | 129.0 | 129.0 | 168.0 |
| **245** | User 246 | 22 | 114.0 | 228.0 | 104.0 | 84.0 | 168.0 |
| | | | .0 | 178.0 | 104.0 | 158.0 | 174.0 |
| | | | .0 | 149.0 | 139.0 | 144.0 | 213.0 |
| **248** | User 249 | 20 | 143.0 | 149.0 | 139.0 | 159.0 | 143.0 |

249 rows × 7 columns

Saved successfully!

```
1 df["Theatre"].fillna("Not present", inplace = True)
2 df
```

| | User Id | Sports | Religious | Nature | Theatre | Shopping | Picnic |
|---|---|---|---|---|---|---|---|
| 1 | User 2 | 2 | 62.0 | NaN | Not present | 60.0 | 68.0 |

Filling a null values using replace() method

```
1 df[10:25]
```

| | User Id | Sports | Religious | Nature | Theatre | Shopping | Picnic |
|---|---|---|---|---|---|---|---|
| **10** | User 11 | 3 | 107.0 | 54.0 | 64.0 | 103.0 | 94.0 |
| **11** | User 12 | 3 | 103.0 | 60.0 | 63.0 | 102.0 | 93.0 |
| **12** | User 13 | 3 | 64.0 | 82.0 | 82.0 | 75.0 | 69.0 |
| **13** | User 14 | 3 | 93.0 | 54.0 | 74.0 | 103.0 | 69.0 |
| **14** | User 15 | 3 | 63.0 | 82.0 | 81.0 | 78.0 | 69.0 |
| **15** | User 16 | 3 | 82.0 | 79.0 | 75.0 | 75.0 | 82.0 |
| **16** | User 17 | 5 | NaN | 131.0 | 103.0 | 54.0 | 86.0 |
| **17** | User 18 | 5 | 56.0 | 124.0 | 108.0 | 56.0 | 85.0 |
| **18** | User 19 | 4 | 85.0 | 67.0 | 111.0 | 65.0 | 72.0 |
| **19** | User 20 | 4 | 83.0 | 83.0 | 65.0 | 114.0 | 102.0 |
| **20** | User 21 | 4 | 93.0 | 82.0 | 79.0 | 79.0 | 90.0 |
| **21** | User 22 | 4 | 105.0 | 52.0 | 75.0 | 113.0 | 78.0 |
| **22** | User 23 | 5 | 69.0 | 118.0 | 74.0 | 66.0 | 101.0 |
| **23** | User 24 | 4 | 71.0 | 123.0 | 64.0 | NaN | 102.0 |
| **24** | User 25 | 5 | 88.0 | 94.0 | 81.0 | 79.0 | 91.0 |

Saved successfully!

```
1 df.replace(to_replace = np.nan, value = -99)
```

| | User Id | Sports | Religious | Nature | Theatre | Shopping | Picnic |
|---|---|---|---|---|---|---|---|
| **0** | User 1 | 2 | 77.0 | 79.0 | 69.0 | 68.0 | 95.0 |
| **1** | User 2 | 2 | 62.0 | 99.0 | Not present | 69.0 | 68.0 |

Using interpolate() function to fill the missing values using linear method.

```
df.interpolate(method ='linear', limit_direction ='forward')
```

| | User Id | Sports | Religious | Nature | Theatre | Shopping | Picnic |
|---|---|---|---|---|---|---|---|
| **0** | User 1 | 2 | 77.0 | 79.0 | 69.0 | 68.0 | 95.0 |
| **1** | User 2 | 2 | 62.0 | 88.0 | Not present | 69.0 | 68.0 |
| **2** | User 3 | 2 | 50.0 | 97.0 | 87.0 | 50.0 | 75.0 |
| **3** | User 4 | 2 | 68.0 | 77.0 | 95.0 | 72.5 | 61.0 |
| **4** | User 5 | 2 | 98.0 | 54.0 | Not present | 95.0 | 86.0 |
| **...** | ... | ... | ... | ... | ... | ... | ... |
| **244** | User 245 | 18 | 139.0 | 148.0 | 129.0 | 129.0 | 168.0 |
| **245** | User 246 | 20 | 134.0 | 228.0 | 104.0 | 84.0 | 168.0 |
| **246** | User 247 | 20 | 124.0 | 178.0 | 104.0 | 158.0 | 174.0 |
| **247** | User 248 | 20 | 133.0 | 149.0 | 139.0 | 144.0 | 213.0 |
| **248** | User 249 | 20 | 143.0 | 149.0 | 139.0 | 159.0 | 143.0 |

249 rows × 7 columns

Saved successfully!

Dropping missing values using dropna()

Drop rows with at least one Nan value

```
df.dropna()
```

| | User Id | Sports | Religious | Nature | Theatre | Shopping | Picnic |
|---|---|---|---|---|---|---|---|
| **0** | User 1 | 2 | 77.0 | 79.0 | 69.0 | 68.0 | 95.0 |
| **2** | User 3 | 2 | 50.0 | 97.0 | 87.0 | 50.0 | 75.0 |
| **4** | User 5 | 2 | 98.0 | 54.0 | Not present | 95.0 | 86.0 |
| **6** | User 7 | 3 | 64.0 | 85.0 | 82.0 | 73.0 | 69.0 |

Dropping rows if all values in that row are missing.

| | ... | ... | ... | ... | ... | ... | ... |
|---|---|---|---|---|---|---|---|

```
1 df.dropna(how = 'all')
```

| | User Id | Sports | Religious | Nature | Theatre | Shopping | Picnic |
|---|---|---|---|---|---|---|---|
| **0** | User 1 | 2 | 77.0 | 79.0 | 69.0 | 68.0 | 95.0 |
| **1** | User 2 | 2 | 62.0 | NaN | Not present | 69.0 | 68.0 |
| **2** | User 3 | 2 | 50.0 | 97.0 | 87.0 | 50.0 | 75.0 |
| **3** | User 4 | 2 | 68.0 | 77.0 | 95.0 | NaN | 61.0 |
| **4** | User 5 | 2 | 98.0 | 54.0 | Not present | 95.0 | 86.0 |
| | | | ... | ... | ... | ... | ... |
| **244** | User 245 | 18 | 139.0 | 148.0 | 129.0 | 129.0 | 168.0 |
| **245** | User 246 | 22 | 114.0 | 228.0 | 104.0 | 84.0 | 168.0 |
| **246** | User 247 | 20 | 124.0 | 178.0 | 104.0 | 158.0 | 174.0 |
| **247** | User 248 | 20 | 133.0 | 149.0 | 139.0 | 144.0 | 213.0 |
| **248** | User 249 | 20 | 143.0 | 149.0 | 139.0 | 159.0 | 143.0 |

249 rows × 7 columns

Saved successfully!

```
1 new_data = df.dropna(axis = 0, how ='any')
2 new_data
```

| | User Id | Sports | Religious | Nature | Theatre | Shopping | Picnic |
|---|---|---|---|---|---|---|---|
| **0** | User 1 | 2 | 77.0 | 79.0 | 69.0 | 68.0 | 95.0 |
| **2** | User 3 | 2 | 50.0 | 97.0 | 87.0 | 50.0 | 75.0 |
| **6** | User 7 | 3 | 64.0 | 85.0 | 82.0 | 73.0 | 69.0 |

compare sizes of data frames so that we can come to know how many rows had at least 1 Null value

```
1 print("Old data frame length:", len(df))
2 print("New data frame length:", len(new_data))
3 print("Number of rows with at least 1 NA value: ", (len(df)-len(new_data)))
```

```
Old data frame length: 249
New data frame length: 230
Number of rows with at least 1 NA value:  19
```

4. Encoding categorical data

Saved successfully!

```
1 df["Sports"].value_counts()
```

```
12    35
18    33
8     29
6     26
22    22
4     19
5     18
14    18
20    17
3     11
25     8
2      5
10     5
16     3
Name: Sports, dtype: int64
```

5. Splitting the dataset into the Training set and Test set

```
1 y= df.Sports
```

```
1 x=df.drop('Sports',axis=1)
```

```
1 x.head()
```

```
2 y.head()
    0    2
    1    2
    2    2
    3    2
    4    2
    Name: Sports, dtype: int64
```

we are using the split ratio of 80:20. The 20% testing data set is represented by the 0.2 at the end.

```
1 sklearn.model_selection.train_test_split(df, test_size=None, train_size=None, random_s
2 x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2)
```

```
1 print("shape of original dataset :", df.shape)
2 print("shape of input - training set", x_train.shape)
3 print("shape of output - training set", y_train.shape)
4 print("shape of input - testing set", x_test.shape)
5 print("shape of output - testing set", y_test.shape)
```

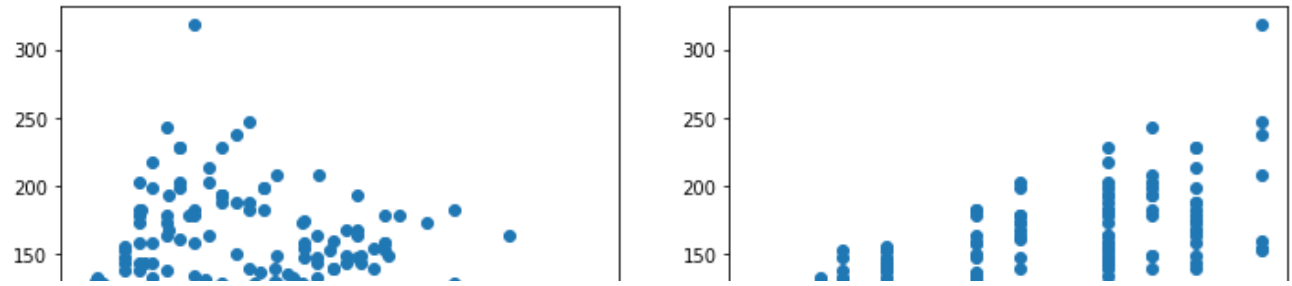
```
    shape of original dataset : (249, 7)
                                199, 6)
    Saved successfully!         (199,)
                                0, 6)
    shape of output - testing set (50,)
```

6. Feature Scaling

Feature Scaling or Standardization: It is a step of Data Pre Processing that is applied to independent variables or features of data. It basically helps to normalize the data within a particular range. Sometimes, it also helps in speeding up the calculations in an algorithm.

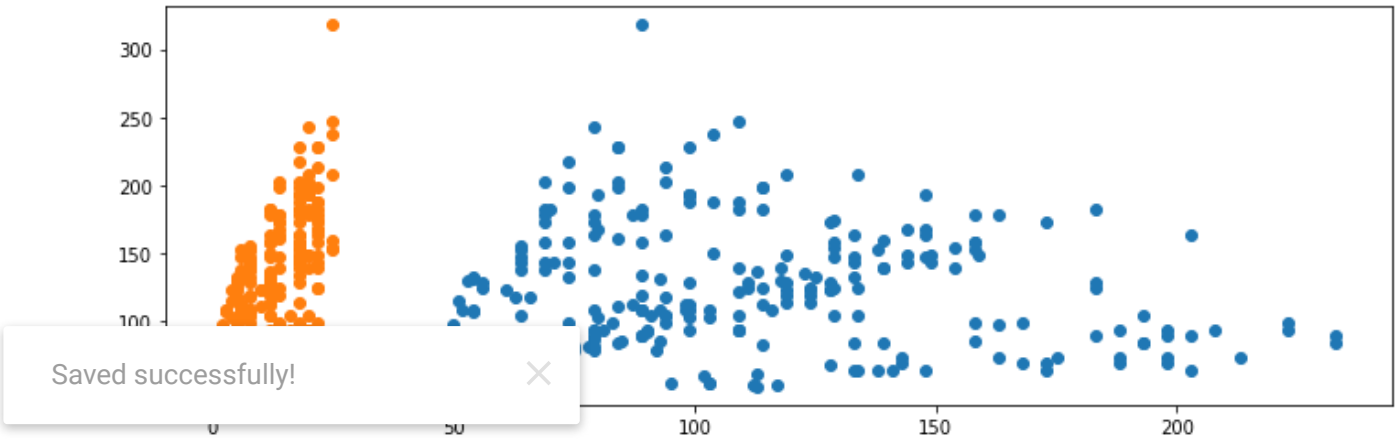
Mostly the Fit method is used for Feature scaling

```
1 x = df[['Shopping', 'Sports']].values
2 y = df['Nature'].values
3
4 fig, ax = plt.subplots(ncols=2, figsize=(12, 4))
5
6 ax[0].scatter(x[:,0], y)
7 ax[1].scatter(x[:,1], y)
8
9 plt.show()
```



```
1 fig, ax = plt.subplots(figsize=(12, 4))
2
3 ax.scatter(x[:,0], y)
4 ax.scatter(x[:,1], y)
```

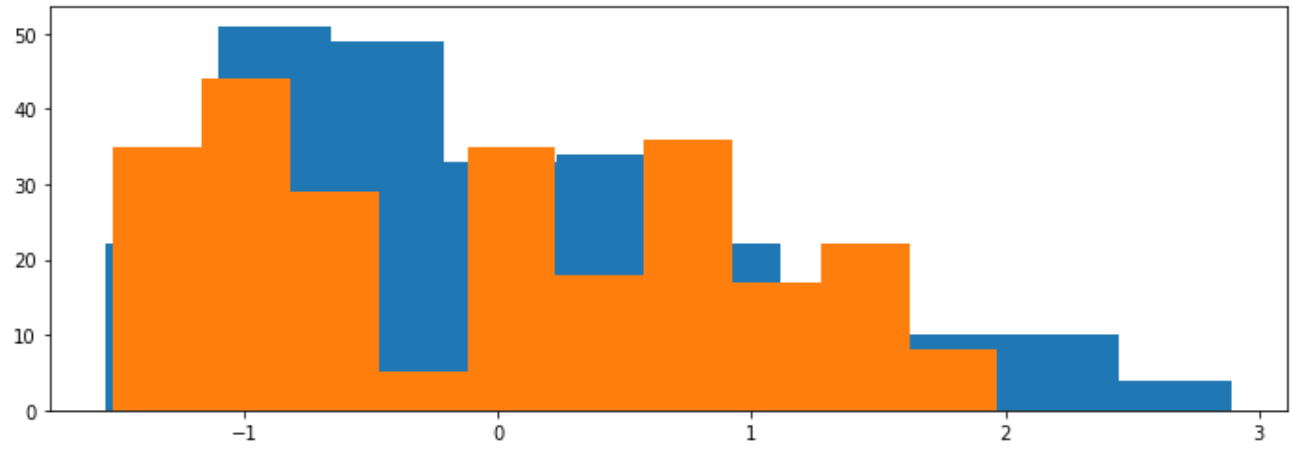
```
<matplotlib.collections.PathCollection at 0x7fe356e2fdd0>
```



Saved successfully!

```
1 fig, ax = plt.subplots(figsize=(12, 4))
2
3 scaler = StandardScaler()
4 x_std = scaler.fit_transform(x)
5
6 ax.hist(x_std[:,0])
7 ax.hist(x_std[:,1])
```

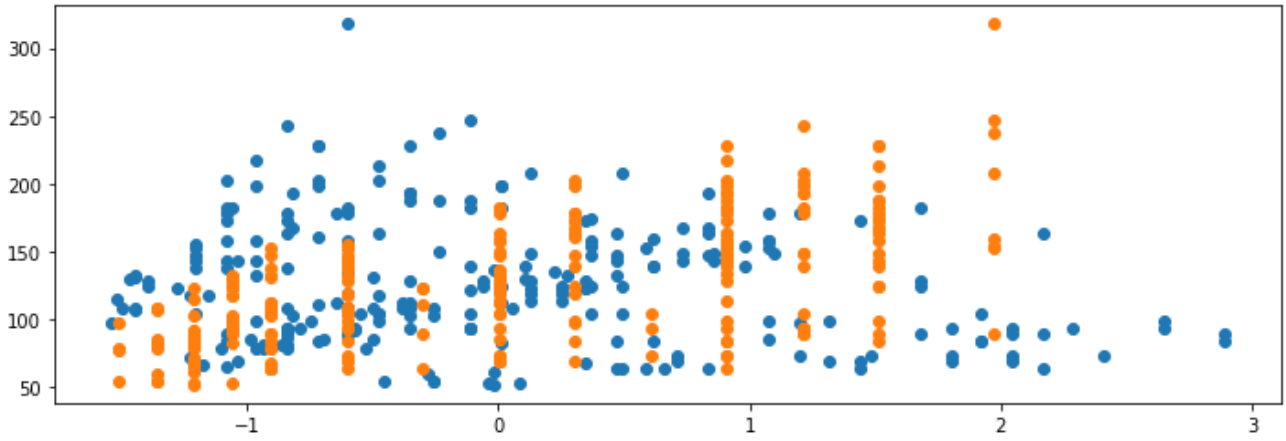
```
(array([35., 44., 29.,  5., 35., 18., 36., 17., 22.,  8.]),
 array([-1.51259241, -1.1642765 , -0.81596058, -0.46764467, -0.11932876,
         0.22898715,  0.57730306,  0.92561897,  1.27393488,  1.62225079,
         1.97056671]),
 <a list of 10 Patch objects>)
```



```
1   fig, ax = plt.subplots(figsize=(12, 4))
2
```

```
3  scaler = StandardScaler()
4  x_std = scaler.fit_transform(x)
5
6  ax.scatter(x_std[:,0], y)
7  ax.scatter(x_std[:,1], y)
```

```
<matplotlib.collections.PathCollection at 0x7fe356deb210>
```




Saved successfully!

0s completed at 11:26 PM